Operating-System Operations and System Protection

Lecture Notes — Week 2

Dual-Mode Operation, Privileged Instructions, and the Timer

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

October 17, 2026

Concepts That Click

2.1 The Control Problem

Reading. Silberschatz et al., Ch. 1 §1.5, pp. 20–23 [1]; Stallings, Ch. 3 §3.5, p. 140 [2].

A student logs in to a shared laboratory machine and runs a three-token program: `while(1);`. The condition is the constant 1, so it is always true; the body is empty; the processor therefore executes the test and the branch forever. The program performs no input, produces no output, issues no system call, and never exits. On a machine with one core, two programs cannot both be "running" at the same instant: the CPU executes one instruction stream at a time. The prediction seems unavoidable — the machine should freeze. It does not. The instructor, working from another terminal, can still list the process and kill it. This chapter explains both facts: why a runaway program cannot monopolize the machine, and why the kernel can always take the CPU back.

The explanation is not that the operating system watches from outside. The operating system (OS) is not a remote controller; it is a program that must run on the same CPU as the code it supervises. Four questions organize what follows. How does the CPU know who is running? The **mode bit** answers it (Section 2.2). What may user code not do? **Privileged instructions** answer it. How does the OS get the CPU back? The **timer** and preemption answer it (Section 2.3). How does a program ask for help? The **system-call gate** answers it (Section 2.4). Every answer is a hardware mechanism, not a software convention: protection only works because the hardware enforces it.

2.1.1 The OS Is a Program Too

The kernel uses the same CPU, the same registers, and the same physical memory as every user program. It cannot run "in the background" while user code runs, because the CPU executes one instruction stream at a time. The kernel therefore runs only when an **event** transfers control to it: a system call, an interrupt, or an exception. Between events it is not executing at all.

That fact creates the central design problem. The hardware must be able to tell, at every instant, whether the current instruction stream is *trusted* kernel code or *untrusted* user code. It must run untrusted code at full speed, yet guarantee that the trusted kernel can always regain control:

> *Build a machine that can run untrusted code at full speed, yet guarantee that the trusted kernel can always regain control.*

Every mechanism in this chapter is an answer to that sentence.

2.1.2 What Goes Wrong Without a Boundary

Without a boundary, four failures follow immediately. A buggy pointer writes over another process's memory — or over the kernel itself — because nothing checks the address. A malicious program disables interrupts, and the timer can no longer fire. A program reprograms the timer never to expire, then loops forever. A program issues raw disk commands, bypassing the file system and every access-control check.

The lesson is blunt: no boundary means no isolation, no multiplexing, and no trust. The rest of this chapter is the mechanism that creates the boundary.

2.2 Dual-Mode Operation

Reading. Silberschatz et al., Ch. 1 §1.5, pp. 20–23 [1].

Protection begins with a single hardware bit. The bit records which of two worlds the CPU is currently executing in, and the instruction set itself changes depending on its value.

Definition (Dual-mode operation). *A hardware* ***mode bit*** *records the current privilege level. The kernel runs in* ***kernel mode*** *(mode bit* $= 0$*); user programs run in* ***user mode*** *(mode bit* $= 1$*). Some instructions are legal only when the bit is* 0*.*

The two modes differ in what they may execute and touch. Kernel mode carries the full instruction set, access to all memory and all devices, and the ability to change the mode bit. User mode carries a restricted instruction set, access only to the process's own address space, and no direct way to change the bit. The restriction is not a convention that well-behaved programs respect; it is enforced by the hardware on every instruction. The mode bit is *hardware state*, not an ordinary memory variable [1].

2.2.1 Who May Execute What?

The instruction set is partitioned accordingly. Table 2.1 lists the two halves.

Privileged (kernel only)	Unprivileged (user or kernel)
• set or clear the mode bit	• arithmetic and logic
• disable or enable interrupts	• loads and stores to the process's own memory
• load the timer	• branches and function calls
• issue raw I/O instructions	• read the clock (not set it)
• change the page-table base	• execute a trap instruction
• halt the machine	

Table 2.1: The privilege partition. A privileged instruction executed in user mode traps to the kernel rather than succeeding silently.

Two entries deserve comment. Loading the timer is privileged because user control of the timer would make preemption optional: the program could simply set the next expiry beyond its own lifetime. Changing the page-table base is privileged because it would otherwise let a process redirect address translation to another process's frames. The privilege boundary is what makes those operations safe to expose at all.

The last unprivileged entry is the important one: a user program may execute a trap instruction. That instruction is the only legal door into the kernel, and Section 2.4 returns to it. A privileged instruction executed in user mode does not silently fail. It traps to the kernel, which normally terminates the process.

Concepts That Click

2.2.2 The Mode Transition Graph

Figure 2.1 shows the complete set of legal transitions between the two modes.

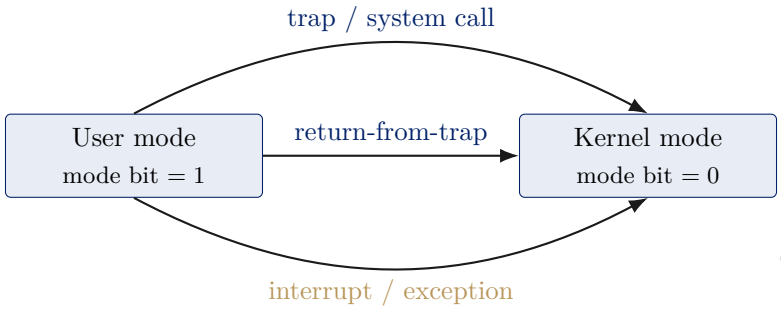


Figure 2.1: The mode transition graph. Entry into the kernel is always a controlled hardware event; the return path restores the saved user context and the user mode bit. Cf. Silberschatz et al., Ch. 1 §1.5, pp. 20–23 [1].

Entry into the kernel is always a **controlled** hardware event. A trap or system call is deliberate; an interrupt or exception is forced. In both cases the hardware, not user code, performs the mode switch and selects the entry point. The return path restores the saved user context and the user mode bit. The kernel does not "call" user code; it returns to it.

2.2.3 The Mode Bit Is Hardware, Not a Variable

The bit must be hardware for a simple reason. If it were stored in ordinary memory, a user program could simply write `0` into it and become the kernel. The instruction that changes the bit is itself privileged, so user code cannot execute it. The hardware defines exactly which events may switch modes: interrupts, exceptions, and traps. This small, hardware-enforced boundary is the foundation of the **trusted computing base** — the set of components the system must trust.

The question is worth asking directly, because it exposes the whole design: why can't the mode bit live in ordinary memory, and what attack becomes possible if it does? Suppose the bit were a word in the user's address space. The program stores `0` into it. It can then disable interrupts, reprogram the timer, or overwrite the page tables. The hardware would no longer have a trusted state to return to. The bit must therefore live outside the address space that user code can write.

> **Consequence.** Protection is not a policy the OS can choose to enforce; it is a property the hardware guarantees. The OS decides *what* to protect, but the mode bit decides *whether* it can.

2.2.4 Worked Trace: One `write` Crosses the Boundary

Example 2.2.1 follows a single library call across the boundary and back.

Example 2.2.1 (One `write` Crosses the Boundary)**.** *User code calls the C library* ***`write(1, buf, n)`****.*

1. *The library places the system-call number and the arguments in registers.*
2. *It executes a* ***trap*** *instruction (`syscall` on x86-64).*

Concepts That Click

3. *Hardware switches to kernel mode, saves the user program counter (PC) and flags, and jumps to the trap handler.*
4. *The kernel validates the arguments, performs the I/O, and stores a return value.*
5. *A* ***return-from-trap*** *restores the user context and the user mode bit.*
6. *The library returns the result; on failure it sets* `errno`*, the C library's error variable.*

Two details are easy to miss. First, the mode switch is a hardware event, not a function call: user code does not choose the kernel's entry address, and it cannot return to an arbitrary instruction inside the kernel. Second, the return value and `errno` are set by the library after the kernel returns; the kernel reports success or failure through the trap's return value. Lecture 3 opens this path up step by step.

Dual-mode operation protects memory and devices. It does not protect time: a user program can loop forever without executing a single privileged instruction. The next section supplies the missing mechanism.

2.3 The Timer and Preemption

Reading. Silberschatz et al., Ch. 1 §1.5, pp. 20–23 [1]; Stallings, Ch. 3 §3.5, p. 140 [2].

2.3.1 The Loop That Never Leaves User Mode

The runaway program is a different kind of problem from the ones dual-mode operation solves. `while(1);` executes only unprivileged instructions. It never traps, never calls the OS, and never touches a device. Every instruction it executes is legal in user mode. If nothing external interrupts it, the kernel never runs again, and the machine is not corrupted — it is simply monopolized.

The OS needs an event that fires *without* the user program's cooperation. That event is the **timer interrupt**.

2.3.2 The Timer Is the External Event

Definition (Timer interrupt). *An asynchronous event raised by a hardware timer when its count reaches zero. It switches the CPU into kernel mode and transfers control to the kernel's timer handler, regardless of what user code is doing.*

The mechanism has five steps. A hardware timer counts down independently of the instruction stream. When it reaches zero, it raises a **timer interrupt**. The interrupt switches the CPU to kernel mode and jumps to the kernel's timer handler. The handler saves the running process's context and calls the scheduler. Loading the timer is a **privileged** instruction, so user code cannot set it.

The timer is a device, not a software alarm. If user code could reload it, preemption would be optional — and a malicious program would opt out.

2.3.3 A Quantum on the Timeline

Figure 2.2 places one scheduling interval, called a quantum, on a timeline.

Definition (Quantum). *The fixed interval of CPU time that the scheduler grants one process before the timer interrupt forces a re-evaluation. In this chapter each process receives a* 10 *ms quantum.*

Concepts That Click

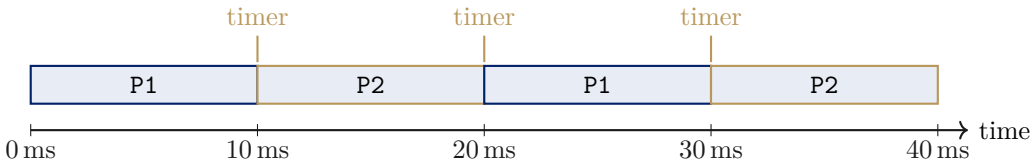


Figure 2.2: Two processes sharing one CPU in 10 ms quanta. The timer fires at 10, 20, and 30 ms, even when the running process is still runnable. Cf. Silberschatz et al., Ch. 1 §1.5, pp. 20–23 [1].

Each process receives one 10 ms quantum. The timer fires at the end of every quantum, even if the process is still runnable. The scheduler decides who runs next; the timer only guarantees that the decision happens. In Figure 2.2, `P1` and `P2` alternate every 10 ms, with timer interrupts at 10, 20, and 30 ms.

2.3.4 Timer plus Scheduler Equals Preemption

Definition (Preemption). *The kernel forcibly takes the CPU away from a running process at a timer interrupt, saves its context, and may schedule a different process.*

Cooperative scheduling makes yielding voluntary: a process yields when it chooses to, and one infinite loop stops the system. Preemptive scheduling makes yielding unnecessary: the timer fires, and the kernel keeps control of the CPU whether or not the process wanted to give it up. The difference is not politeness; it is who holds the power to decide.

The choice of quantum is a trade-off. A large quantum wastes response time: an interactive process waits longer before it next runs. A tiny quantum wastes CPU time on context switches. The next example makes the second cost quantitative.

2.3.5 Worked Example: What Does a Quantum Cost?

Example 2.3.1 (The Overhead of a Context Switch)**.** *A context switch costs* 12 μs*. The quantum is* Q*. What fraction of CPU time is spent switching?*

Let Q *be measured in microseconds. The CPU completes* $1/Q$ *quanta per microsecond, hence* $10^6/Q$ *quanta per second, and one context switch occurs at each quantum boundary. The switching time per second is therefore*

$$\left(\frac{10^6}{Q}\right)(12\ \mu s) = \frac{12 \times 10^6}{Q}\ \mu s.$$

As a fraction of one second (10^6 μs*), the overhead is* $12/Q$*. Equivalently,*

$$\textit{overhead} = \frac{12\ \mu s}{Q},$$

provided Q *is expressed in microseconds.*

Quantum Q	***Switches per second***	***Overhead***
100 μs	10,000	120,000 $\mu s = 12\%$
1 ms	1,000	12,000 $\mu s = 1.2\%$
10 ms	100	1,200 $\mu s = 0.12\%$

Concepts That Click

For $Q = 100$ μs, there are 10,000 switches per second; at 12 μs each, that is 120,000 $\mu s = 0.12$ s, or 12% of the second. For $Q = 1$ ms the same calculation gives 1,000 switches and 12,000 $\mu s = 0.012$ s, or 1.2%. For $Q = 10$ ms it gives 100 switches and 1,200 $\mu s = 0.0012$ s, or 0.12%. The formula is the point: the fixed 12 μs cost is divided by a larger Q as the quantum grows. This is why the quantum cannot simply be made as small as possible.

2.3.6 Why Not a One-Microsecond Quantum?

If a small quantum improves response time, why not set the quantum to 1 μs? At 1 μs the switch overhead alone is 12 μs/1 μs = 12 = 1200%: the CPU spends twelve times as long switching as executing. Two further costs compound the problem. Cache and translation lookaside buffer (TLB) state are destroyed on every switch, so the next process starts cold. The scheduler itself consumes CPU time to choose the next process. Real systems therefore choose a quantum that balances responsiveness against overhead, often in the 1–100 ms range.

2.4 Protection in Depth

Reading. Silberschatz et al., Ch. 1 §1.5, pp. 20–23 [1]; Stallings, Ch. 3 §3.5, p. 140, and §3.6, p. 143 [2].

The mode bit and the timer protect the CPU. A complete boundary also protects memory and the devices.

2.4.1 Three Things the Kernel Protects

Figure 2.3 summarizes the three resources and the mechanism that guards each one.



Figure 2.3: The three resources the kernel protects and the hardware mechanism behind each. Cf. Silberschatz et al., Ch. 1 §1.5, pp. 20–23 [1]; Stallings, Ch. 3 §3.5, p. 140 [2].

CPU time is protected by the timer, which forces the kernel to run. Memory is protected by the memory-management unit (MMU), which checks every user access against the process's legal address range. I/O is protected because devices are reachable only through privileged instructions and kernel drivers.

2.4.2 Memory Protection

Three organizations are common, and all three enforce the same rule at different granularities. With **contiguous allocation**, base and limit registers bound the process, and the hardware checks every address against them. With **paging**, the page table maps only the process's legal pages, and a reference to an unmapped page traps to the kernel. With **segmentation**, each segment has its own base and limit, and the hardware checks the offset against the limit.

The pattern is the same in every case: user code never sees a physical address. Every access is translated and checked by hardware, and a violation becomes a **trap**, not a silent write. Details belong to Unit III (Lectures 20–25).

Concepts That Click

2.4.3 I/O Protection

All raw I/O instructions are privileged. A user program cannot talk to a disk, a network card, or the timer directly. The user-level `read` and `write` calls are system calls: the kernel validates the buffer, the device, and the count. It then invokes the device driver, which programs the controller. The utility `cat` never touches the disk controller; it asks the kernel, and the kernel decides whether the request is legal.

This matters because direct device access would let a process bypass file permissions, corrupt another process's data, or hang the device for everyone. Privileging the instruction is what makes the kernel's validation unavoidable.

2.4.4 Protection versus Security

Table 2.2 separates two terms that are often used interchangeably.

Protection	**Security**
Internal mechanisms among processes	External threats and attackers
memory isolation, privilege levels, access control	authentication, malware defense, network firewalls
Enforced by the OS and hardware	Enforced by policy, cryptography, and administrative practice

Table 2.2: Protection is the mechanism; security is the policy that uses it. Cf. Stallings, Ch. 3 §3.6, p. 143 [2].

Protection is a *mechanism*; security is a *policy* that uses the mechanism. Protection concerns internal mechanisms among processes: memory isolation, privilege levels, and access control. Security concerns external threats and attackers: authentication, malware defense, and network firewalls. Protection is enforced by the OS and hardware; security is enforced by policy, cryptography, and administrative practice. A perfect mode bit does not help if the administrator grants every user root access [2].

2.4.5 The System-Call Gate

Figure 2.4 shows the controlled doorway between the two modes.

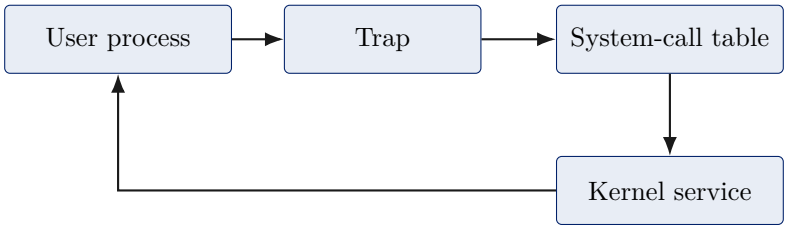


Figure 2.4: The system-call gate: a trap dispatches through the system-call table to a kernel service and returns through the same controlled entry point. Cf. Silberschatz et al., Ch. 2 §2.3 [1]; Bovet and Cesati, Ch. 10, pp. 398–419 [3].

Concepts That Click

The trap is the **only legal door** from user mode into the kernel. The kernel validates every argument and every pointer before acting. The return path restores user mode; the kernel never returns to arbitrary user code. Figure 2.4 shows the path: the user process traps, the trap dispatches through the system-call table to a kernel service, and the service returns through the same gate. Lecture 3 opens this door and walks through it.

2.4.6 The Legal Path for `cat`

As a check, suppose all I/O instructions are privileged and ask how `cat` reads a file. The legal path has three steps.

1. `cat` calls the library `read`, which traps into the kernel.
2. The kernel checks the file descriptor, buffer, and count, then asks the device driver to perform the transfer.
3. The driver programs the controller; the kernel returns the data and restores user mode.

Nothing in that sequence requires `cat` to execute a privileged instruction. The privilege lives in the kernel; the user program only asks.

2.5 Putting It Together

Reading. Silberschatz et al., Ch. 1 §1.5, pp. 20–23 [1]; Stallings, Ch. 3 §3.5, p. 140 [2].

Protection is not one mechanism. It is a loop: the timer fires, the kernel runs, the scheduler chooses, and user code resumes — or traps when it asks for something it cannot do.

2.5.1 The Full Control Loop

Figure 2.5 draws the loop.

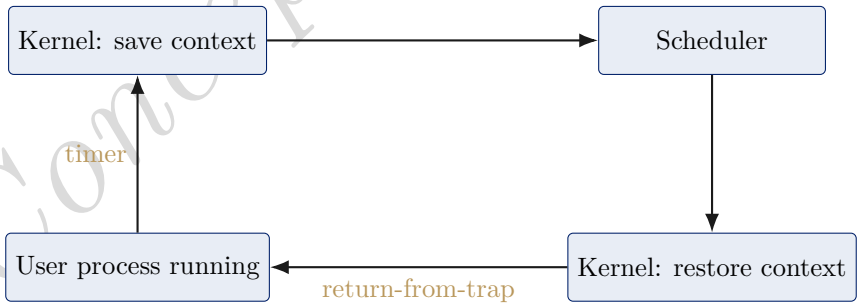


Figure 2.5: The full control loop: a timer interrupt transfers control to the kernel, the scheduler chooses, and a return-from-trap resumes user code. Cf. Silberschatz et al., Ch. 1 §1.5, pp. 20–23 [1]; Stallings, Ch. 3 §3.5, p. 140 [2].

The loop runs on every timer tick and every blocking system call. The kernel is *event-driven*: it runs because something happened, not because it is "always on". The user process cannot see the loop; it only sees time passing.

Concepts That Click

2.5.2 Worked Trace: Stopping the Runaway Process

Example 2.5.1 (Stopping the Runaway Process)**.** *`P1` enters `while(1);` in user mode at $t = 0$. The timeline unfolds as follows.*

1. *$t = 0$: `P1` enters `while(1);` in user mode.*
2. *$t = 10$ ms: the timer fires; hardware switches to kernel mode.*
3. *The kernel saves `P1`'s PC and registers in its process control block (PCB).*
4. *The scheduler picks `P2`; the kernel restores `P2`'s context and returns to user mode.*
5. *$t = 20$ ms: the timer fires again; the scheduler may pick `P1` or a third process.*
6. *If `P1` must be stopped, the kernel can deliver a signal or terminate it — it already has control.*

At no step is `P1` asked to cooperate. The timer is external, the save is kernel work, and the scheduler's choice is made in kernel mode.

2.5.3 Common Misconceptions

Table 2.3 pairs the four most common wrong statements with their corrections.

Wrong	Right
"Dual-mode is a software convention."	The mode bit and privileged instructions are hardware.
"The timer is a software alarm."	The timer is a hardware device loaded by the kernel.
"User code can disable interrupts if it is fast enough."	Interrupt control is privileged; attempts trap.
"The OS runs continuously in the background."	The kernel is event-driven, entered through traps and interrupts.

Table 2.3: Four misconceptions about protection, and the hardware fact that corrects each one.

2.5.4 Takeaways

- **Dual-mode operation** uses a hardware mode bit to separate trusted kernel code from untrusted user code.
- **Privileged instructions** protect the mode bit, the timer, the page tables, and the devices.
- **The timer** is the external event that lets the kernel preempt a runaway process.
- **The system-call gate** is the controlled entry point; the kernel validates every request.
- Protection is a *hardware-enforced* boundary, not a promise the OS makes.

Concepts That Click

2.5.5 Looking Ahead: System Calls

Today's question was how the OS stays in control — mode bit, privileged instructions, timer, and the trap gate. The next lecture asks what happens *inside* the gate: how a program names a service, how parameters travel, and what kinds of calls exist. The reading is Silberschatz et al., Ch. 2 §2.3–2.4 [1], and Bovet and Cesati, Ch. 10 [3], for the concrete Linux view.

2.6 Summary

The operating system is a program that must run on the same CPU as the code it supervises, so its authority cannot come from standing outside the machine. It comes from a hardware boundary. A mode bit records whether the CPU is executing trusted kernel code or untrusted user code, and privileged instructions — changing the mode bit, disabling interrupts, loading the timer, issuing raw I/O, changing the page-table base, halting the machine — are legal only in kernel mode. A privileged instruction executed in user mode traps to the kernel instead of succeeding. Because the bit lives in hardware, outside the address space user code can write, a program cannot promote itself by storing a value into memory.

Dual-mode operation protects memory and devices, but not time. A process that executes only unprivileged instructions can loop forever without ever trapping, so the kernel needs an event that fires without the process's cooperation. That event is the timer interrupt: a hardware timer counts down independently, switches the CPU to kernel mode when it expires, and hands control to the kernel's timer handler. The handler saves the running process's context, the scheduler chooses the next process, and a return-from-trap resumes user code. Preemption is the result: the kernel forcibly takes the CPU away at a timer interrupt rather than waiting for the process to yield. The quantum controls the trade-off; with a 12 μs context switch, the overhead is 12 μs/Q, so shrinking the quantum eventually makes switching cost more than execution.

A complete boundary also covers memory and I/O. Base and limit registers, page tables, or segment registers ensure that every user access is translated and checked, and a violation traps rather than writing silently. Raw I/O is privileged, so user programs reach devices only through system calls that the kernel validates. Protection is the mechanism; security is the policy that uses it. The system-call gate is the single controlled entry point, and the kernel returns to user mode only through the paired return path. In the running thread, the runaway `while(1);` process is stopped not by persuading it to yield but by the timer interrupt, which gives the kernel control and lets it save the process, schedule another, and terminate the runaway if necessary — all without the process's cooperation.

Concepts That Click

Self-Check

1. What does each value of the mode bit mean, and name two instructions that are legal only in kernel mode.

2. Why must the mode bit be hardware rather than an ordinary memory variable? Describe the attack that becomes possible if it is not.

3. A process executes only unprivileged instructions forever. Name the mechanism that returns control to the kernel, and outline the sequence.

4. A context switch costs 12 μs. Compute the switching overhead for $Q = 2$ ms and for $Q = 250$ μs.

5. Distinguish protection from security, and trace `cat`'s read in three steps.

Exercises

1. A user program executes an instruction that attempts to disable interrupts. Describe exactly what happens, using the terms *privileged instruction*, *trap*, and *mode bit*. Why is the outcome not the same as an ordinary function call?

2. Classify each event as an interrupt, an exception, or a trap, and state which direction it moves the mode bit: (a) the timer expires; (b) a program executes `syscall`; (c) a user program executes a privileged instruction. Which of these can the user program prevent?

3. A context switch costs 12 μs. Compute the fraction of CPU time spent switching for $Q = 250$ μs and for $Q = 2$ ms. Which quantum is the better choice for an interactive system, and what does the other cost?

4. Three processes `P1`, `P2`, and `P3` are ready. `P1` enters `while(1);` at $t = 0$; the scheduler is round-robin with a 10 ms quantum and switch cost is ignored. Describe the execution from $t = 0$ to $t = 40$ ms, and state every instant at which the kernel has control and could terminate `P1`.

5. A process is confined by base register `0x10000` and a limit register holding the size 64 KB. Which of these user addresses are legal: `0x10000`, `0x1FFFF`, `0x20000`? What happens on an illegal access, and why is that outcome preferable to a silent write?

Solutions

1. The instruction is privileged, so the hardware checks the current mode before executing it. The process is in user mode (mode bit = 1), so the instruction cannot execute. The hardware raises an exception that traps to the kernel; the kernel's handler normally terminates the process. The outcome differs from a function call because the transfer is forced by hardware at a fixed entry point: the user program does not choose the kernel address, does not run with kernel privileges, and cannot decide whether the kernel returns to it.

2. (a) The timer expiry is an interrupt: asynchronous and hardware-raised, moving the CPU from user mode to kernel mode (or interrupting the kernel itself). (b) `syscall` is a trap: software-initiated by the running program, deliberately moving user mode to kernel mode. (c) Executing a privileged instruction in user mode is an exception: the hardware rejects the instruction and traps to the kernel. The user program can initiate (b) but cannot prevent the kernel from handling it; it cannot prevent (a), which is external; and it cannot make (c) succeed, because the hardware, not the program, decides legality.

3. With $Q = 250$ μs, there are $1/250$ μs $= 4{,}000$ switches per second; at 12 μs each, the switching time is 48,000 μs $= 0.048$ s, or 4.8% of one second. With $Q = 2$ ms, there are 500 switches per second, giving 6,000 μs $= 0.006$ s, or 0.6%. The 2 ms quantum is the better choice for an interactive system because it keeps overhead at 0.6% while still giving each process the CPU frequently; the 250 μs quantum improves responsiveness further but spends 4.8% of the CPU switching, and also destroys more cache and TLB state.

4. From $t = 0$ to $t = 10$ ms, `P1` runs its infinite loop. At $t = 10$ ms the timer fires, the kernel saves `P1`'s context, and the scheduler picks `P2`; `P2` runs from 10 to 20 ms. At $t = 20$ ms the timer fires, the kernel saves `P2`, and the scheduler picks `P3`; `P3` runs from 20 to 30 ms. At $t = 30$ ms the timer fires, and the scheduler picks `P1` again; `P1` runs from 30 to 40 ms, when the next timer interrupt arrives. The kernel has control at $t = 10$, 20, 30, and 40 ms, so it can deliver a signal or terminate `P1` at any of those instants even though `P1` never yields.

Concepts That Click

5. The limit register holds the size, 64 KB = `0x10000` bytes, so the legal range is `0x10000` $\leq a <$ `0x20000`. `0x10000` is the first legal byte and `0x1FFFF` is the last; both are legal. `0x20000` is exactly one byte past the end and is illegal. The hardware rejects the access and traps to the kernel, which normally terminates the process. A silent write would corrupt whichever memory happened to lie beyond the limit — possibly another process's data or the kernel itself — so trapping converts an invisible corruption into a controlled, attributable fault.

Further Reading (Page-Cited)

- A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 8e — Ch. 1 §1.5 (dual-mode operation, timer, protection, pp. 20–23).
- W. Stallings, *Operating Systems: Internals and Design Principles*, 6e — Ch. 3 §3.5 (execution of the OS, p. 140), §3.6 (security issues, p. 143).
- D. P. Bovet and M. Cesati, *Understanding the Linux Kernel*, 3e — Ch. 4 (interrupts and exceptions, pp. 131–188), Ch. 10 (system calls, pp. 398–419), for the concrete x86/Linux view.

Page-verified via each book's `index.md` under `master_supporting_docs/CS502/supporting_books/`.

References

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 8th ed., John Wiley and Sons — Ch. 1 (pp. 3–47), Ch. 2 (pp. 49–89). *(Page-cited; index built 2026-10-14.)*

[2] W. Stallings, *Operating Systems: Internals and Design Principles*, 6th ed., Prentice Hall — Ch. 3 §3.5 (p. 140), §3.6 (p. 143). *(Page-cited; index built 2026-10-14.)*

[3] D. P. Bovet and M. Cesati, *Understanding the Linux Kernel*, 3rd ed., O'Reilly Media, 2006 — Ch. 4 (interrupts and exceptions, pp. 131–188), Ch. 10 (system calls, pp. 398–419). *(Page-cited; index built 2026-10-14.)*

Concepts That Click